

Batched Dense Optical Flow in PyTorch

批量稠密光流的 PyTorch 实现与基准测试

TV-L1 & DeepFlow: OpenCV CPU vs. CUDA vs. pure-torch reimplementation

T. Long

tlong32@jhu.edu

August 2026

Middlebury pairs 480×640 · NVIDIA L40 (SLURM, ilps-cn119) · 48-core CPU login node

- ▶ Dense optical flow is a standard **preprocessing step at scale**: video understanding, motion features, temporal augmentation, dataset curation — millions of frame pairs, not one.
- ▶ Classic accurate methods (TV-L1, DeepFlow) are **variational**: hundreds of small iterations per pair $\Rightarrow$ slow on CPU, awkward on GPU.
- ▶ What matters for preprocessing is **batch throughput** (pairs/second over a whole dataset), not single-pair latency.
- ▶ Goal: benchmark OpenCV CPU vs. OpenCV CUDA vs. our **batched pure-PyTorch reimplementations**, on identical inputs and parameters.

中文要点 稠密光流常作为大规模视频处理的预处理步骤：要处理的是数百万帧对，而非单帧。经典变分方法（TV-L1、DeepFlow）迭代次数多、单对速度慢；预处理场景真正重要的是批量吞吐量。我们在相同输入与参数下对比 OpenCV CPU / OpenCV CUDA / 自研批量化 PyTorch 实现。

Data & hardware

- ▶ Middlebury training pairs, 480×640 grayscale (also downscaled 240×320); ground-truth flow for EPE.
- ▶ GPU: NVIDIA L40 (142 SMs), SLURM node `ilps-cn119`.
- ▶ CPU: 48-core login node.
- ▶ Env: `uv-managed`; `opencv-contrib-python-headless`, PyTorch.

Algorithm × backend matrix

	cv CPU	cv CUDA	torch
TV-L1	✓	✓*	✓
DeepFlow	✓	none	✓
Farneback	✓	✓*	—

*only with a custom CUDA build of OpenCV — pip wheels ship **without** CUDA.

中文要点 数据：Middlebury 训练集（ 480×640 ，含真值光流，用于计算端点误差 EPE）。硬件：NVIDIA L40（142 个 SM，SLURM 节点）对比 48 核 CPU 登录节点。注意：pip 安装的 OpenCV 不带 CUDA，需要自行编译；DeepFlow 在 OpenCV 中没有任何 CUDA 版本。

$$E(\mathbf{u}) = \int_{\Omega} \lambda |\rho(\mathbf{u}(\mathbf{x}))| + |\nabla u_1| + |\nabla u_2| \, d\mathbf{x}$$

with the image residual linearized around the current warp $\mathbf{u}^0$:

$$\rho(\mathbf{u}) = l_1(\mathbf{x} + \mathbf{u}^0) + \langle \nabla l_1(\mathbf{x} + \mathbf{u}^0), \mathbf{u} - \mathbf{u}^0 \rangle - l_0(\mathbf{x})$$

- ▶ L^1 data term: robust to outliers/occlusions; TV term: allows sharp flow discontinuities at motion boundaries.
- ▶ Both terms non-smooth $\Rightarrow$ solved via **duality**, not gradient descent.
- ▶ Params used (OpenCV defaults): $\lambda = 0.15$, $\theta = 0.3$, $\tau = 0.25$, 5 scales (step 0.8), 5 warps, up to 10×30 primal-dual iterations.

中文要点 能量 = 全变差正则项 + 鲁棒 L^1 数据项； ρ 是在当前解 $\mathbf{u}^0$ 处线性化的亮度残差。 L^1 对遮挡/离群点鲁棒，TV 项保留运动边界的不连续性。两项都不可微，因此用对偶方法求解而非梯度下降。

TV-L1: Duality Splitting & Iterations 对偶分裂与迭代

Auxiliary field $\mathbf{v}$ decouples the two terms (θ -coupling):

$$\min_{\mathbf{u}, \mathbf{v}} \int_{\Omega} |\nabla u_1| + |\nabla u_2| + \frac{1}{2\theta} \|\mathbf{u} - \mathbf{v}\|^2 + \lambda |\rho(\mathbf{v})| \, d\mathbf{x}$$

1) **Pointwise thresholding** (closed form in $\mathbf{v}$):

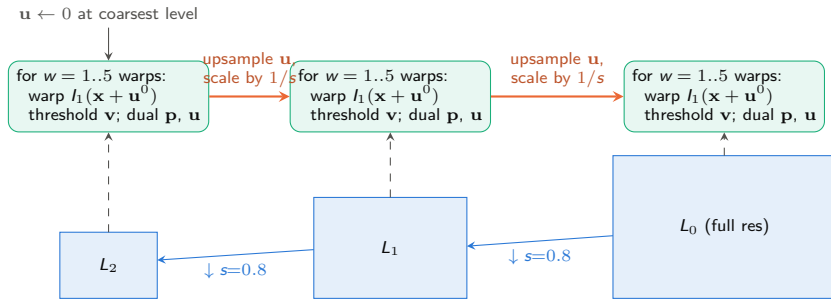
$$\mathbf{v} = \mathbf{u} + \begin{cases} \lambda\theta \nabla I_1 & \text{if } \rho(\mathbf{u}) < -\lambda\theta |\nabla I_1|^2 \\ -\lambda\theta \nabla I_1 & \text{if } \rho(\mathbf{u}) > \lambda\theta |\nabla I_1|^2 \\ -\rho(\mathbf{u}) \nabla I_1 / |\nabla I_1|^2 & \text{if } |\rho(\mathbf{u})| \leq \lambda\theta |\nabla I_1|^2 \end{cases}$$

2) **TV proximal step** via dual variables $\mathbf{p}_d$ (Chambolle), for $d = 1, 2$:

$$u_d = v_d + \theta \operatorname{div} \mathbf{p}_d, \quad \mathbf{p}_d^{n+1} = \frac{\mathbf{p}_d^n + \frac{\tau}{\theta} \nabla u_d^{n+1}}{1 + \frac{\tau}{\theta} |\nabla u_d^{n+1}|} \quad (\text{projection onto } |\mathbf{p}_d| \leq 1), \quad \tau \leq 1/4$$

中文要点 引入辅助变量 $\mathbf{v}$ 将两项解耦：第一步对每个像素做闭式阈值化 更新 $\mathbf{v}$ ；第二步用对偶变量 $\mathbf{p}$ 做 TV 近端步（分母即到单位球的投影）。两步都是纯逐像素运算——非常适合张量化。

Coarse-to-Fine Pyramid + Warping 由粗到细金字塔与图像变形



Linearization ρ is only valid for *small* displacements $\Rightarrow$ estimate coarse motion on small images, then warp l_1 by the upsampled flow and re-linearize at each finer level.

中文要点 线性化残差 ρ 只对小位移成立：先在低分辨率估计大运动，再逐层上采样光流（数值乘 $1/s$ ），用它对 l_1 做 warp 后重新线性化。每层做 5 次 warp，每次 warp 内做数百次原始-对偶迭代。

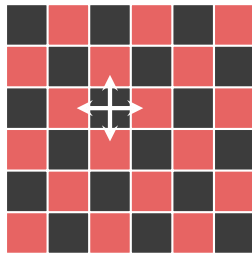
$$E(\mathbf{w}) = \int_{\Omega} \left[\begin{aligned} &\delta \Psi(|I_2(\mathbf{x} + \mathbf{w}) - I_1(\mathbf{x})|^2) && \text{brightness constancy} \\ &+ \gamma \Psi(\|\nabla I_2(\mathbf{x} + \mathbf{w}) - \nabla I_1(\mathbf{x})\|^2) && \text{gradient constancy} \\ &+ \alpha \Psi(\|\nabla u\|^2 + \|\nabla v\|^2) \end{aligned} \right] d\mathbf{x} \quad \text{smoothness}$$

- ▶ Full DeepFlow adds a matching term βE_{match} driving the flow toward **DeepMatching** correspondences.
- ▶ **OpenCV's** `createOptFlow_DeepFlow` implements only the variational part — no DeepMatching ($\beta = 0$).
- ▶ Params (OpenCV defaults): $\alpha = 1$, $\delta = 0.5$, $\gamma = 5$, pyramid downscale 0.95 ($\sim 40+$ levels), 5 fixed-point iterations, 25 SOR iterations, $\omega = 1.6$.

中文要点 能量含亮度守恒 (权重 δ)、梯度守恒 (γ) 与平滑项 (α)，均用鲁棒罚函数 $\Psi(s^2) = \sqrt{s^2 + \varepsilon^2}$ 。完整 DeepFlow 另有 DeepMatching 匹配项；OpenCV 只实现了变分部分 (无匹配项)。

DeepFlow: Fixed-Point + Red-Black SOR 不动点迭代与红黑 SOR

- ▶ Euler–Lagrange equations are nonlinear in $\mathbf{w} = (u, v)$: **fixed-point linearization** — freeze Ψ' weights and image derivatives at $\mathbf{w}^k$, solve for the increment $d\mathbf{w}$.
- ▶ Each linearization yields a large **sparse linear system** (5-point stencil per pixel, 2 coupled channels).
- ▶ Solved by **SOR** ($\omega = 1.6$): each pixel update mixes its own residual with already-updated neighbors.
- ▶ **Red-black ordering**: a red pixel's stencil touches only black pixels, so *all red pixels update simultaneously*, then all black — two fully parallel half-sweeps.



A red update reads only **black** neighbors $\Rightarrow$ each half-sweep is one *vectorized masked update*.

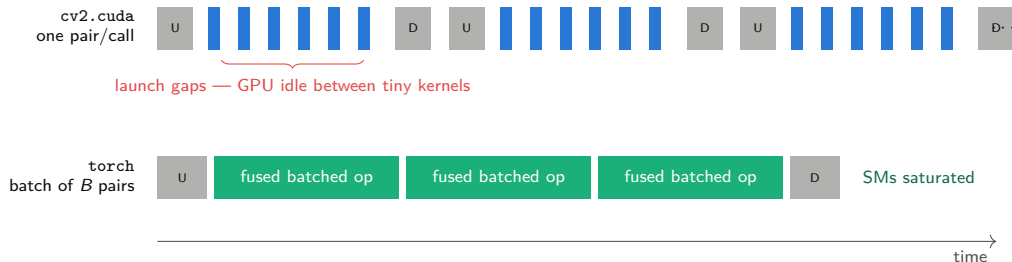
中文要点 外层不动点迭代固定 Ψ' 权重得到稀疏线性方程组，内层用 SOR ($\omega = 1.6$) 求解；红黑排序下红格只依赖黑格，红/黑各自可全并行更新——即两次带掩码的张量化更新。

Why OpenCV GPU Is Limited Here OpenCV GPU 路线的局限

1. **Packaging:** pip opencv-python wheels ship *without* CUDA $\Rightarrow$ custom source build per cluster/CUDA version — a real operational burden.
2. **API is one-pair-at-a-time:** cv2.cuda takes a single GpuMat pair. A 480×640 (let alone 240×320) problem cannot fill an L40's **142 SMs**; upload/download + kernel-launch overhead dominates.
3. **Hundreds of tiny sequential iterations:** TV-L1 ≈ 5 scales $\times 5$ warps $\times$ up to 300 primal-dual steps $\Rightarrow$ thousands of small kernels per pair — *launch-bound*, GPU mostly idle.
4. **DeepFlow: no CUDA version exists in OpenCV at all**; and a naive (lexicographic) SOR sweep is inherently sequential — it must be re-ordered (red-black) to parallelize.

中文要点 (1) pip 版 OpenCV 不含 CUDA, 须自行编译; (2) cv2.cuda 一次只处理 一对图像, 小图无法喂饱 L40 的 142 个 SM, 上传/下载与核启动开销占主导; (3) TV-L1 有成千上万个微小串行核, 属于启动开销受限; (4) DeepFlow 根本没有 OpenCV CUDA 版本, 且朴素 SOR 天生串行。

Launch-Bound vs. Batched 核启动受限 vs. 批量化



Same total math — but one launch now covers B images' worth of pixels, so arithmetic hides the launch latency and fills the GPU.

中文要点 上：cv2.cuda 每对图像触发一串微小核函数，核间空隙里 GPU 空转。下：批量张量把 B 对图像合成一次大核启动——总计算量相同，但每次启动的工作量足以摊薄启动延迟并填满所有 SM。

Our Approach: Pure-Torch, Batched 纯 PyTorch 批量化实现

`torch_flow.py (calc_flow_tv11)` · `torch_deepflow.py (calc_flow_deepflow)`

- ▶ **Batched end to end:** (B, H, W) image pairs $\rightarrow (B, 2, H, W)$ flows; every step is a vectorized tensor op over the whole batch — no Python loop over pairs.
- ▶ **Warping** = `grid_sample` (bilinear, one call for the batch).
- ▶ **Red-black SOR / primal-dual** steps as masked tensor updates: two half-sweeps = two fused elementwise ops.
- ▶ **Same code on CPU and CUDA** — just `.to(device)`; no custom OpenCV build.
- ▶ **Composable** with a torch pipeline: `DataLoader` feeding, `torch.no_grad()` inference, `torch.compile` for kernel fusion.

中文要点 输入 (B, H, W) 帧对、输出 $(B, 2, H, W)$ 光流，全流程张量化：warp 用 `grid_sample`，红黑 SOR / 原始-对偶迭代化为带掩码的融合逐元素运算。同一份代码在 CPU 与 GPU 上直接运行，且天然融入 torch 数据管线（`DataLoader`、`no_grad` 推理、`torch.compile`）。

Runtime Results 运行时间结果

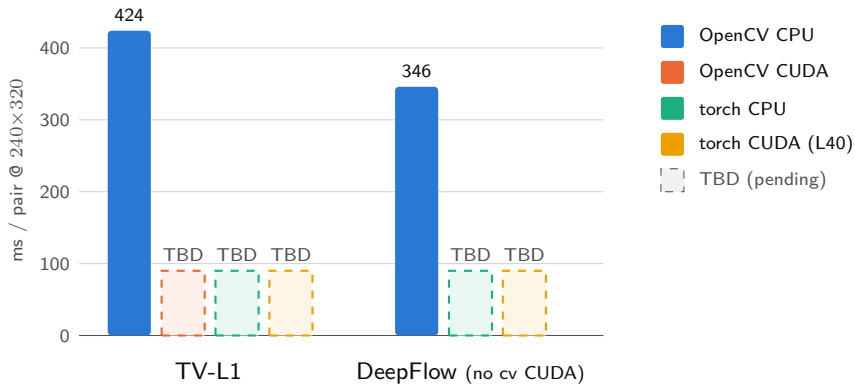
Per-pair wall time, identical parameters across backends. *TBD* = measurement in progress.

Algorithm	Size	cv CPU	cv CUDA	torch CPU	torch CUDA (L40)
TV-L1	240×320	424 ms (2.4 FPS)	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
DeepFlow	240×320	346 ms (2.9 FPS)	— (none)	<i>TBD</i>	<i>TBD</i>
Farneback	480×640	84 ms	—	—	—

- ▶ CPU numbers from the 48-core login node — but these algorithms **barely parallelize** in OpenCV: Farneback single-thread vs 48-thread is 85 vs 84 ms. Cores don't help; batching must.
- ▶ torch CUDA column is **batched** throughput (time per pair = batch time / B).

中文要点 OpenCV CPU 已确认：TV-L1 每对 424 毫秒、DeepFlow 346 毫秒（ 240×320 ）。48 核几乎不加速（Farneback 单线程 85 ms vs 48 线程 84 ms）——多核帮不上忙，必须靠批量化。torch CPU/GPU 数据测量中（表中 TBD 一处修改即可全局填入）。

Runtime Chart 运行时间对比图



Dashed bars: measurements pending; heights will update automatically from the placeholder macros.

中文要点 实心柱为已确认的 OpenCV CPU 结果；虚线柱为待测的 torch CPU / torch CUDA 与 OpenCV CUDA 结果，数值填入宏后柱高自动更新。

Accuracy: Endpoint Error 精度：端点误差

Mean EPE on Middlebury pairs at 240×320 (pixels with GT magnitude $> 10^9$, i.e. “unknown”, excluded).

Algorithm	Backend	EPE vs. GT	EPE vs. OpenCV ref.
TV-L1	OpenCV CPU	0.149	(reference)
TV-L1	torch	<i>TBD</i>	<i>TBD</i>
DeepFlow	OpenCV CPU	0.084	(reference)
DeepFlow	torch	<i>TBD</i>	<i>TBD</i>

- ▶ DeepFlow’s variational part is already notably more accurate than TV-L1 here (0.084 vs 0.149 EPE).
- ▶ “EPE vs. OpenCV ref.” measures **faithfulness of the reimplementaion** (target $\ll$ EPE vs. GT); small differences expected from interpolation/boundary choices.

中文要点 以 Middlebury 真值计算平均端点误差 (EPE) : DeepFlow (0.084) 明显优于 TV-L1 (0.149)。“vs. OpenCV” 一系列衡量我们复现实现与 OpenCV 的一致性, 目标是远小于对真值的误差; 插值与边界处理会带来微小差异。torch 数据待填。

Conclusions

- ▶ Variational flow is iteration-heavy and core-count-insensitive; the viable route to throughput is **batching**, which OpenCV's APIs (CPU or CUDA) do not offer.
- ▶ A pure-torch rewrite gives batching, GPU support without custom OpenCV builds, one code path for CPU/GPU, and pipeline composability.
- ▶ Red-black reordering is the key trick that makes SOR-style solvers tensor-friendly.

Practical notes

- ▶ Env: uv project; use `opencv-contrib-python-headless` on compute nodes (no GUI libs).
- ▶ GPU runs via SLURM: `srun -p gpu -w ilps-cn119 --gres=gpu:1 --cpus-per-task=8 --mem=16G uv run --no-sync python benchmark.py`
- ▶ Login node has no GPU: CPU backends only; benchmark script auto-skips unavailable backends.

中文要点 结论：变分光流迭代多、对核数不敏感，吞吐量的正确路径是批量化；纯 torch 重写免去 OpenCV 自编译、CPU/GPU 一套代码、可组合进现有管线。实用：用 uv 管理环境，计算节点装 headless 版 OpenCV，GPU 作业经 SLURM 提交至 ilps-cn119。

1. C. Zach, T. Pock, H. Bischof. *A Duality Based Approach for Realtime TV-L1 Optical Flow*. DAGM Symposium on Pattern Recognition, 2007.
2. P. Weinzaepfel, J. Revaud, Z. Harchaoui, C. Schmid. *DeepFlow: Large Displacement Optical Flow with Deep Matching*. ICCV 2013.
3. J. Sánchez Pérez, E. Meinhardt-Llopis, G. Facciolo. *TV-L1 Optical Flow Estimation*. Image Processing On Line (IPOL), vol. 3, 2013.
4. S. Baker, D. Scharstein, J.P. Lewis, S. Roth, M.J. Black, R. Szeliski. *A Database and Evaluation Methodology for Optical Flow*. IJCV, 2011. (Middlebury benchmark)
5. OpenCV documentation: `cv::optflow::DualTVL1OpticalFlow`,
`cv::optflow::createOptFlow_DeepFlow`, `cv::cuda::OpticalFlowDual_TVL1`.

中文要点 以上为 TV-L1 (Zach 等 2007 ; Sánchez 等 IPOL 2013 实现)、DeepFlow (Weinzaepfel 等 ICCV 2013)、Middlebury 数据集及 OpenCV 文档的出处。